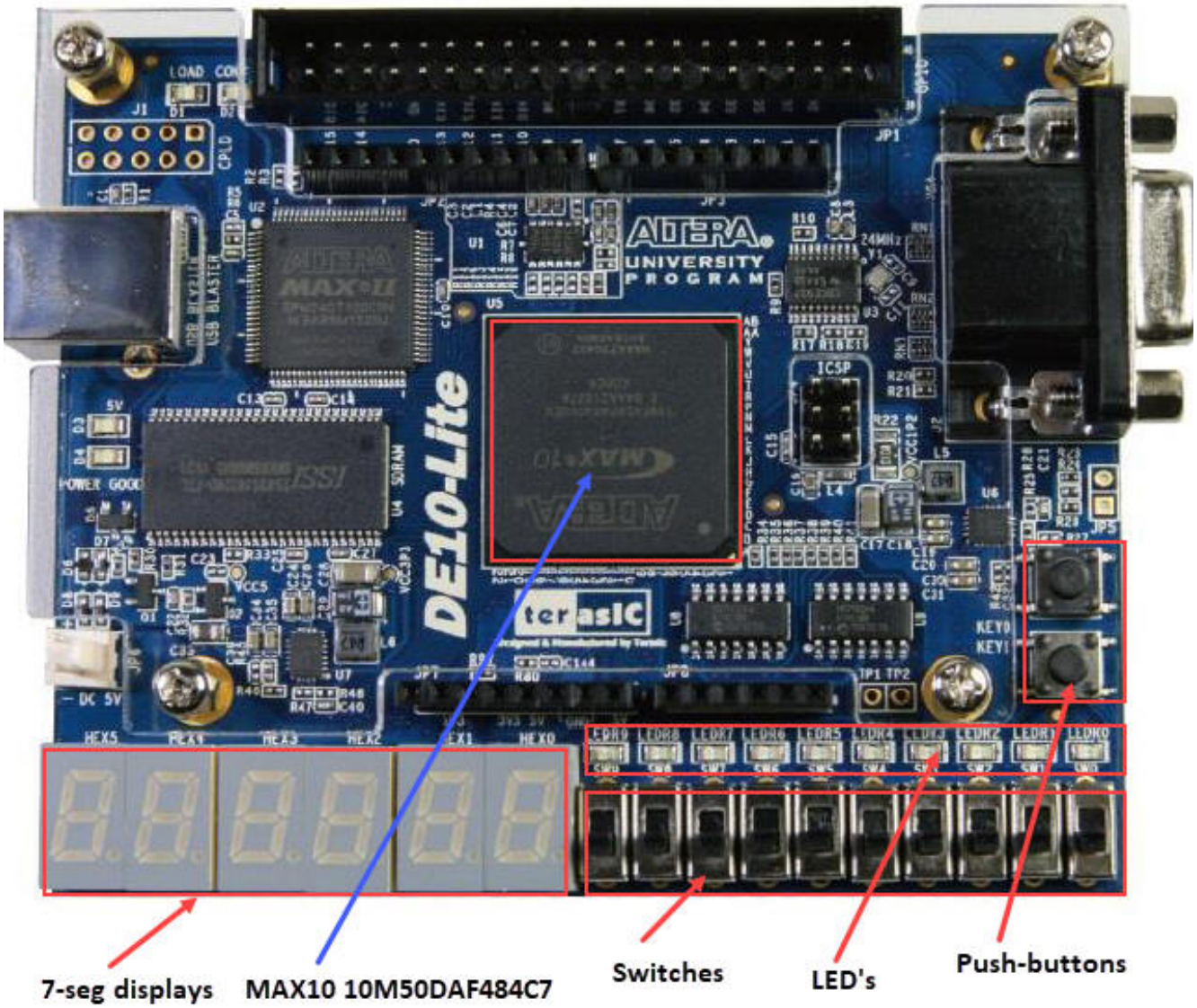
TP1: Implémentation d'un SoC à base de NiosII sur la carte DE10_Lite



System On Chip

Avec le développement des composants programmables de type FPGA à haute intégration, il devient possible d'implémenter logiciellement des systèmes complexes qui autrefois nécessitaient le développement d'une puce spécifique (ASIC = Application-Specific Integrated Circuit). Les deux plus grands fabriquant de FPGA XILINX et ALTERA fournissent aujourd'hui des outils logiciels qui permettent de concevoir des SoPC (Systems on Programmable Chips).

On aborde ici le domaine du CoDesign logiciel et matériel. En effet, on conçoit des systèmes embarqués qui intègrent à la fois des processeurs généralistes capables d'exécuter des programmes (logiciel) et des blocs IP (Intellectual Properties). La conception des éléments logiciels et matériels du système se fait conjointement.

Revenons sur la notion d'IP. Ce sont des blocs fonctionnels généralement complexes qui sont réutilisables et que l'on peut facilement intégrer au SoPC que l'on conçoit. Ils peuvent être de type matériel et dans ce cas ils sont optimisés pour une technologie ou de type logiciel et décrits par un langage de programmation (VHDL, Verilog, SystemVerilog, etc.). Généralement les interfaces avec ces blocs IP sont normalisées. Dans la pratique, différentes sociétés mettent à disposition des IP cores pour différentes applications. Ces composants sont généralement payants et sont souvent cryptés (le code source n'est pas disponible). ALTERA fournit une bibliothèque d'IP cores que l'on peut tester en mode OpenCore plus, c'est-à-dire que le bloc fonctionne tant que la carte FPGA est liée au PC ou il s'arrête de fonctionner au bout d'une Heure.

Il existe également des blocs IP en logiciel libre que l'on trouve sur OpenCores : http://opencores.org/

Pour construire un SoPC, il faut disposer d'une unité de calcul, c'est-à-dire d'un processeur. Il y a de nombreuses possibilités, les cœurs les plus connus étant les cœurs ARM. En ce qui concerne ALTERA nous pouvons disposer du cœur NIOS II gratuitement qui est un processeur 32 bits à architecture RISC (Reduced Instruction Set Computer). Ce processeur s'interface facilement avec des périphériques via le bus propriétaire Avalon-MM (Avalon Memory Mapped).
Le processeur NIOS II est décliné en trois versions dont la puissance dépend des besoins du système à concevoir (cf. Figure 2). La Figure 3 donne les puissances en MIPS (Million of Instructions Per Seconds) pour les différentes versions du processeur NIOS II et de la plateforme FPGA utilisée.

	Nios II /f Fast	Nios II /s Standard	Nios II /e Economy
Pipeline	6 Stage	5 Stage	None
H/W Multiplier & Barrel Shifter	1 Cycle	3 Cycle	Emulated In Software
Branch Prediction	Dynamic	Static	None
Instruction Cache	Configurable	Configurable	None
Data Cache	Configurable	None	None
Logic Usage (Logic Elements)	1400 - 1800	1200 – 1400	600 – 700
Custom Instructions	Up to 256		

Figure 2 : Les 3 versions du processeur NIOS II

Nios II Processor Performance (DMIPS)

Device Family	Nios II /f	Nios II /s	Nios II /e
Std Cell ASIC (90 nm est.)	>500		
Stratix III / Stratix IV FPGAs (prelim)	340	140/150	48
Stratix II FPGAs	250	110	45
Stratix FPGAs	170	80	27
Hardcopy II devices	230	130	50
Hardcopy devices	165	85	27
Cyclone III FPGAs	195	90	30
Cyclone II FPGAs	145	55	18
Cyclone FPGAs	130	52	17

Figure 3 : Performances du processeur NIOS II

Objectifs:

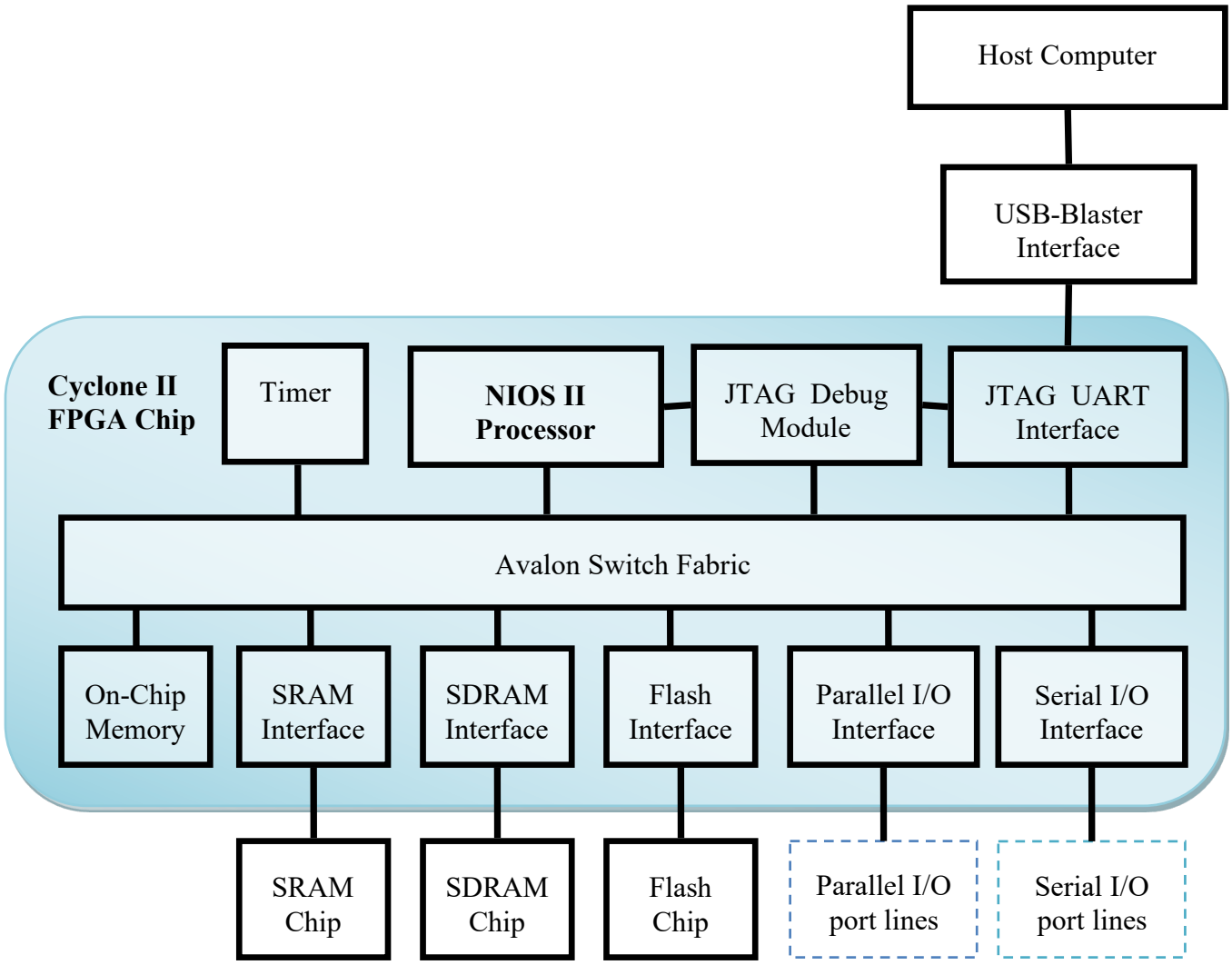
- Familiarisation avec l'outil Platform Designer.
- Création d'un système complet à base du softcore NIOS II.
- Compiler et exécuter du code sur le Nios II.

Dans la première partie de ce TP, vous allez apprendre à utiliser les outils (QuartusPrime et Platform Designer) qui vont vous permettre de créer votre plate-forme hardware en implémentant le processeur Nios II et un ensemble de périphériques de base. Dans une deuxième partie, vous allez utiliser l'outil NiosII Software Build Tools (basé sur Eclipse) qui permet d'écrire et compiler des programmes en C (software) qui vont être exécutés sur cette plate-forme.

Pour tous les TPs, vous allez utiliser la carte d'expérimentation DE10-Lite qui comprend un FPGA MAX10 de chez Altera

Le processeur Nios II est un soft processeur, définit avec un langage de description matérielle, qui peut être implémenté sur les FPGA Intel Altera en utilisant la suite logicielle Quartus Prime. Pour implémenter un système complet, il est nécessaire d'ajouter d'autres éléments au processeur, tel que des mémoires, des interfaces d'entrées/sorties, des temporisateurs (timers) et des interfaces de communications.

Un système complet à base de Nios II peut être implémenté sur la carte DE10-Lite comme le montre la figure ci-dessous.

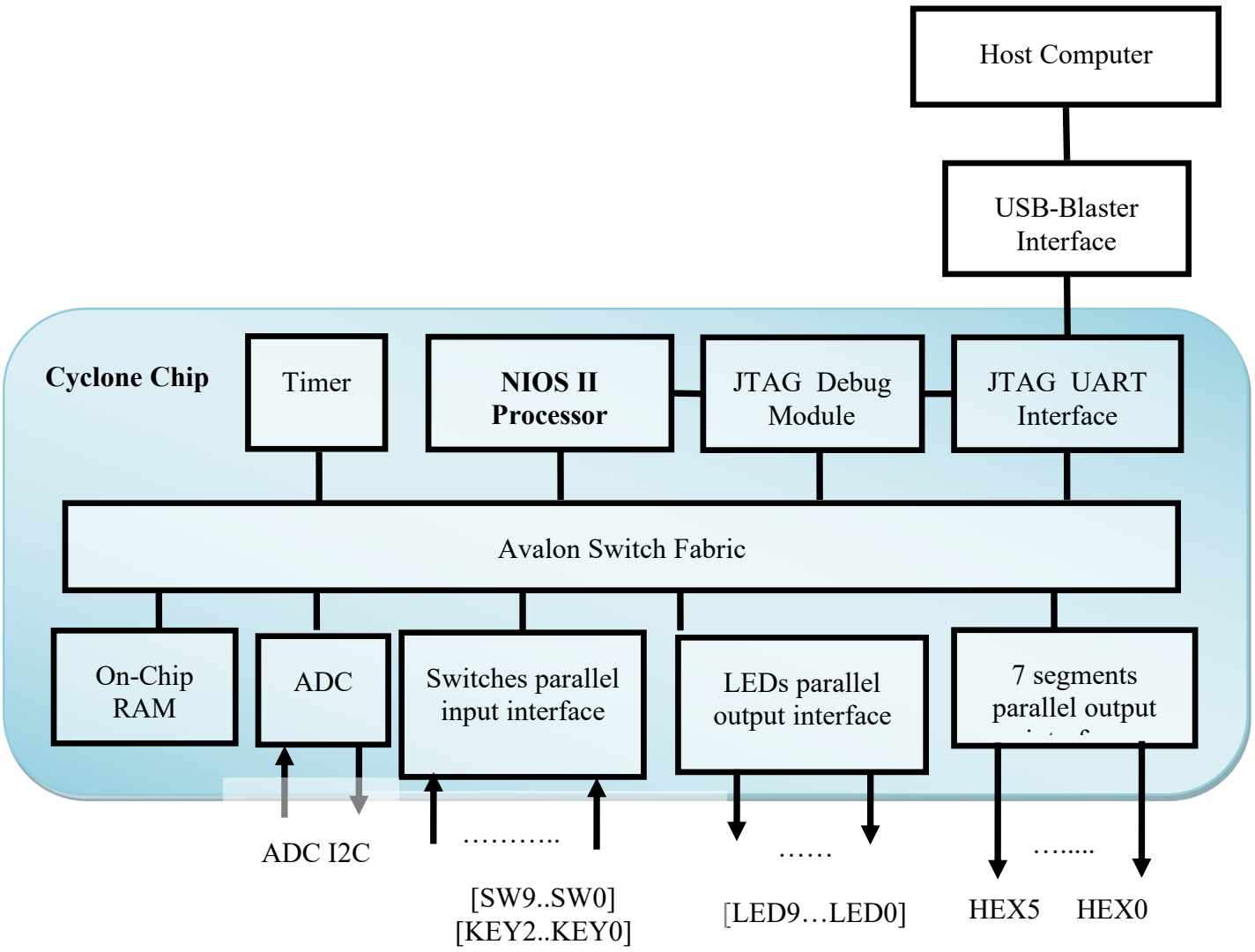


Pour ce premier TP, vous allez uniquement implémenter les périphériques de base qui permettent de réaliser des applications simples telles que :

- Allumer des LEDs séquentiellement,
- Lire les valeurs des interrupteurs et des boutons poussoirs.
- Afficher des valeurs sur les afficheurs 7 segments
- Communiquer avec le PC via une liaison série

Au niveau hardware, pour réaliser ces applications, vous avez besoin d'implémenter les périphériques suivants (comme le montre la figure ci-dessous) :

- Un microprocesseur et sa mémoire.
- Des entrées/sorties d'usages général (GPIO) pour les LEDs, les interrupteurs, les boutons poussoirs et les afficheurs 7 segments.
- D'un temporisateur pour l'affichage séquentiel sur les LEDs
- D'un UART pour les transmissions séries.


Host Computer
USB-Blaster Interface
Cyclone Chip
Timer
NIOS II Processor
JTAG Debug Module
JTAG UART Interface
Avalon Switch Fabric
On-Chip RAM
ADC
Switches parallel input interface
LEDs parallel output interface
7 segments parallel output
ADC I2C
[SW9..SW0]
[KEY2..KEY0]
[LED9…LED0]
HEX5
HEX0

Outils :

- Quartus Prime **18.1**
- Platform Designer (ex-Qsys)
- Nios II Software Build Tools (SBT)

Introduction et objectifs

À l'issue de ce TP, l'étudiant sera capable de :

- Créer un projet Quartus Prime **from scratch** pour la carte DE10-Lite
- Concevoir un **système sur puce (SoC)** simple autour d'un processeur **Nios II** à l'aide de Platform Designer
- Comprendre l'architecture matérielle d'un système Nios II (bus Avalon, adressage mémoire, interruptions)
- Générer le matériel et l'intégrer dans un projet FPGA
- Développer un logiciel embarqué en langage C :
  - sans HAL, par accès direct aux registres (pointeurs `volatile`)
  - avec HAL (drivers fournis par Altera)
- Mettre en œuvre l'ensemble des périphériques du système

L'objectif est de réaliser un système matériel complet autour du processeur **Nios II**, comprenant une mémoire « On-Chip RAM », plusieurs interfaces PIO (Parallel I/O) pour piloter les LED, lire les interrupteurs et les boutons, piloter les afficheurs 7 segments, deux timers, un port **JTAG UART** et un périphérique **System ID**. Dans une deuxième phase on développera du code en langage C pour exploiter ce système, d'abord via des accès mémoire directs (pointeurs `volatile`), puis en utilisant la couche d'abstraction matérielle (HAL). Enfin des exercices supplémentaires permettront de manipuler les timers, d'afficher des valeurs sur les afficheurs 7 segments et de réaliser un **chenillard**.

Partie 1 : Création du système matériel avec Platform Designer

1.1 Configuration du projet Quartus Prime

1. **Créer un nouveau projet** dans Quartus Prime 18.1 : choisir un répertoire de travail et nommer le projet (par exemple `tp_de10_nios_soc`). Sélectionner la carte ou le périphérique **MAX10** correspondant à la DE10-Lite (**10M50DAF484C7G**).
2. Dans le **Pin Planner** de Quartus, importer éventuellement le fichier de contraintes (`.qsf`) fourni avec la carte DE10-Lite, qui assigne chaque port logique (LEDs, interrupteurs, afficheurs…) à la broche adéquate. Les tableaux 3-3 à 3-6 du manuel précisent la correspondance entre les signaux et les broches : par exemple `SW0` est relié à la broche C10, `LED0` à la broche A8, et les lignes des afficheurs 7 segments sont listées de `HEX00` à `HEX46`. L'import d'un fichier `.qsf` évite d'avoir à réaliser ces attributions manuellement.

Voici ci-dessous un **fichier `.qsf` prêt à copier-coller** (pin assignments) correspondant exactement aux ports de ton top-level VHDL (`CLOCK_50`, `KEY`, `SW`, `LEDR`, `HEX0..HEX5`) et **basé sur les tables de pin assignment du DE10-Lite User Manual** (clocks, push-buttons, switches, LEDs, 7-segments).

À enregistrer par exemple sous : `de10lite_pins.qsf`
Puis dans Quartus : **Assignements → Import Assignements (en important le fichier .qsf)**

```
#============================================================
# CLOCK
#============================================================

set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to CLOCK_50
set_location_assignment PIN_P11 -to CLOCK_50

#============================================================
# SEG7
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX0[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX1[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX2[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX3[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX4[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to HEX5[7]
set_location_assignment PIN_C14 -to HEX0[0]
set_location_assignment PIN_E15 -to HEX0[1]
set_location_assignment PIN_C15 -to HEX0[2]
set_location_assignment PIN_C16 -to HEX0[3]
set_location_assignment PIN_E16 -to HEX0[4]
set_location_assignment PIN_D17 -to HEX0[5]
set_location_assignment PIN_C17 -to HEX0[6]
```

```
set_location_assignment PIN_D15 -to HEX0[7]
set_location_assignment PIN_C18 -to HEX1[0]
set_location_assignment PIN_D18 -to HEX1[1]
set_location_assignment PIN_E18 -to HEX1[2]
set_location_assignment PIN_B16 -to HEX1[3]
set_location_assignment PIN_A17 -to HEX1[4]
set_location_assignment PIN_A18 -to HEX1[5]
set_location_assignment PIN_B17 -to HEX1[6]
set_location_assignment PIN_A16 -to HEX1[7]
set_location_assignment PIN_B20 -to HEX2[0]
set_location_assignment PIN_A20 -to HEX2[1]
set_location_assignment PIN_B19 -to HEX2[2]
set_location_assignment PIN_A21 -to HEX2[3]
set_location_assignment PIN_B21 -to HEX2[4]
set_location_assignment PIN_C22 -to HEX2[5]
set_location_assignment PIN_B22 -to HEX2[6]
set_location_assignment PIN_A19 -to HEX2[7]
set_location_assignment PIN_F21 -to HEX3[0]
set_location_assignment PIN_E22 -to HEX3[1]
set_location_assignment PIN_E21 -to HEX3[2]
set_location_assignment PIN_C19 -to HEX3[3]
set_location_assignment PIN_C20 -to HEX3[4]
set_location_assignment PIN_D19 -to HEX3[5]
set_location_assignment PIN_E17 -to HEX3[6]
set_location_assignment PIN_D22 -to HEX3[7]
set_location_assignment PIN_F18 -to HEX4[0]
set_location_assignment PIN_E20 -to HEX4[1]
set_location_assignment PIN_E19 -to HEX4[2]
set_location_assignment PIN_J18 -to HEX4[3]
set_location_assignment PIN_H19 -to HEX4[4]
set_location_assignment PIN_F19 -to HEX4[5]
set_location_assignment PIN_F20 -to HEX4[6]
set_location_assignment PIN_F17 -to HEX4[7]
set_location_assignment PIN_J20 -to HEX5[0]
set_location_assignment PIN_K20 -to HEX5[1]
set_location_assignment PIN_L18 -to HEX5[2]
set_location_assignment PIN_N18 -to HEX5[3]
set_location_assignment PIN_M20 -to HEX5[4]
set_location_assignment PIN_N19 -to HEX5[5]
set_location_assignment PIN_N20 -to HEX5[6]
set_location_assignment PIN_L19 -to HEX5[7]

#============================================================
# KEY
#============================================================
set_instance_assignment -name IO_STANDARD "3.3 V SCHMITT TRIGGER" -to KEY[0]
set_instance_assignment -name IO_STANDARD "3.3 V SCHMITT TRIGGER" -to KEY[1]
set_location_assignment PIN_B8 -to KEY[0]
set_location_assignment PIN_A7 -to KEY[1]

#============================================================
# LED
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[8]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to LED[9]
set_location_assignment PIN_A8 -to LED[0]
set_location_assignment PIN_A9 -to LED[1]
set_location_assignment PIN_A10 -to LED[2]
set_location_assignment PIN_B10 -to LED[3]
set_location_assignment PIN_D13 -to LED[4]
set_location_assignment PIN_C13 -to LED[5]
set_location_assignment PIN_E14 -to LED[6]
set_location_assignment PIN_D14 -to LED[7]
set_location_assignment PIN_A11 -to LED[8]
set_location_assignment PIN_B11 -to LED[9]

#============================================================
# SW
```

```
#============================================================
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[8]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to SW[9]
set_location_assignment PIN_C10 -to SW[0]
set_location_assignment PIN_C11 -to SW[1]
set_location_assignment PIN_D12 -to SW[2]
set_location_assignment PIN_C12 -to SW[3]
set_location_assignment PIN_A12 -to SW[4]
set_location_assignment PIN_B12 -to SW[5]
set_location_assignment PIN_A13 -to SW[6]
set_location_assignment PIN_A14 -to SW[7]
set_location_assignment PIN_B14 -to SW[8]
set_location_assignment PIN_F15 -to SW[9]


set_global_assignment -name PARTITION_NETLIST_TYPE SOURCE -section_id Top
set_global_assignment -name PARTITION_FITTER_PRESERVATION_LEVEL PLACEMENT_AND_ROUTING -section_id Top
set_global_assignment -name PARTITION_COLOR 16764057 -section_id Top
set_instance_assignment -name PARTITION_HIERARCHY root_partition -to | -section_id Top
```

3. **Ouvrir Platform Designer** via le menu *Tools → Platform Designer*. Cet outil permet d'assembler des blocs IP et génère automatiquement les interconnexions (bus Avalon) nécessaires.

1.2. Ajout des composants IP

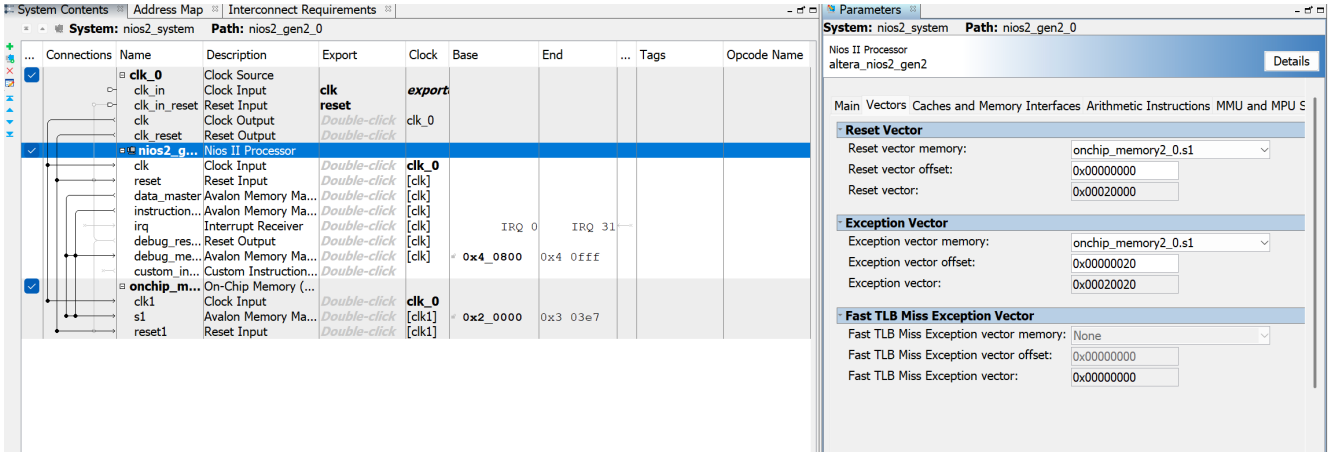
La création du SoC se fait en plusieurs étapes : on ajoute les IP depuis la bibliothèque, on configure leurs paramètres, on les connecte entre eux puis on assigne des adresses et des interruptions.

1.2.1. Nios II et mémoire On-Chip

- Dans l'onglet **IP Catalog**, ouvrir **Processors and Peripherals → Embedded Processors** et ajouter un **Nios II Processor**. Dans l'assistant de configuration, choisir la variante **Nios II/e** (économique) pour cet exemple. Les options par défaut (cache désactivé) conviennent.
- Ajouter une **On-Chip Memory (RAM or ROM) Intel FPGA IP** via **Basic Functions → On-Chip Memory**. Choisir une taille de 128 kB (131 072 Bytes) et décocher l'option « Initialyze memory content ». Cette mémoire hébergera le programme et les données.

  Attention : Ne pas oublier de décocher « Initialyze memory content »

- Configurer ce système minimal en connectant les entrées **clk** et **reset** du processeur et de la mémoire au signaux **clk_0.clk** (horloge de 50 MHz) et **clk_0.clk_reset**.
- Pour la mémoire, relier le port **s1** (Avalon slave) aux ports **data_master** et **instruction_master** du processeur Nios II comme sur la figure ci-dessous.
- Ensuite assignez automatiquement les adresses en allant sur **System → Assign Base Addresses**
- Enfin sélectionnez les vecteurs de la onchip_memory dans l'onglet Vectors du processeur NiosII.



1.2.2. Interfaces PIO pour les périphériques

Les interfaces PIO permettent de lire ou d'écrire des lignes de bits via l'Avalon bus. Elles sont utilisées pour les LED, les interrupteurs, les boutons et les afficheurs 7 segments. La documentation explique que pour une PIO unidirectionnelle, seul le registre **Data** est nécessaire et l'adresse de ce registre est assignée automatiquement par Platform Designer.

1. **LED PIO** : sélectionner **Processors and Peripherals → Peripherals → PIO (Parallel I/O)**, choisir une largeur de **10 bits** et le mode **Output**. Renommer le composant `LED`.
2. **Switch PIO** : ajouter une PIO d'entrée de **10 bits** et renommer `SW`. Les interrupteurs coulissants fournissent un niveau logique haut ou bas.
3. **Buttons PIO** : ajouter une PIO d'entrée de **2 bits** pour les deux boutons poussoirs et renommer `KEY`.
4. **Afficheurs 7 segments (hex_pio)** : la carte DE10-Lite possède 6 afficheurs de 8 bits (7 segments + décimal). Chaque affichage nécessite 8 lignes, soit **48 bits** au total. Créer un PIO de **32 bits** en mode **Output** et renommer `HEX3_0`. Créer un PIO de **16 bits** en mode **Output** et renommer `HEX5_4`

1.2.3. Autres périphériques

- **JTAG UART** : dans **Interface Protocols → Serial**, ajouter un **JTAG UART Intel FPGA IP**. Ne pas modifier les paramètres. Ce noyau permet d'envoyer et de recevoir des caractères via l'interface JTAG. Les instructions du tutorial indiquent de connecter la ligne d'interruption du JTAG UART au processeur en sélectionnant un numéro d'IRQ (par exemple 0)
- **Timers** : ajouter deux instances de **Interval Timer Intel FPGA IP** depuis **Processors and Peripherals → Peripherals**. Le timer dispose de registres de période, de commande et d'état et peut générer des interruptions périodiques. Renommer les timers `timer_0` et `timer_1`.
- **System ID** : ajouter un **System ID Peripheral** via **Basic Functions → Simulation; Debug and Verification → Debug and Performance**. Ce périphérique fournit un identifiant et un champ de timestamp utilisés par le logiciel pour vérifier que le binaire correspond bien au matériel.

1.2.4. Connexions

Les blocs ajoutés apparaissent dans l'onglet **System Contents**. Connecter les signaux comme suit :

- Relier l'entrée **clk** de chaque périphérique au signal `clk_0.clk` (horloge de 50 MHz). Vérifier la fréquence dans les propriétés du composant `clk_0`.
- Relier l'entrée **reset** de chaque périphérique au signal `clk_0.reset`.
- Pour chaque périphérique, relier le port **s1** (Avalon slave) aux ports **data_master** et **instruction_master** du processeur Nios II. Les PIOs, les timers, le JTAG UART et le System ID se connectent uniquement au master de données. La mémoire se connecte aux deux (instructions et données).
- Lignes d'interruptions : assigner des numéros d'IRQ. Par exemple : JTAG UART = 0, timer_0 = 1, timer_1 = 2. Dans Platform Designer, cliquez dans la colonne **IRQ** et entrez le numéro souhaité.

1.2.5. Vecteurs de réinitialisation et d'exception

Pour que le processeur exécute le programme depuis la mémoire on-chip, il faut définir les vecteurs. Dans le composant `nios2` (clic droit → **Edit** → onglet **Vectors**), sélectionner la mémoire on-chip (`onchip_memory_0.s1`) pour le vecteur de réinitialisation et d'exception. La documentation indique que cette opération supprime les messages d'erreur concernant l'assignation des adresses.

1.2.6. Attribution des adresses

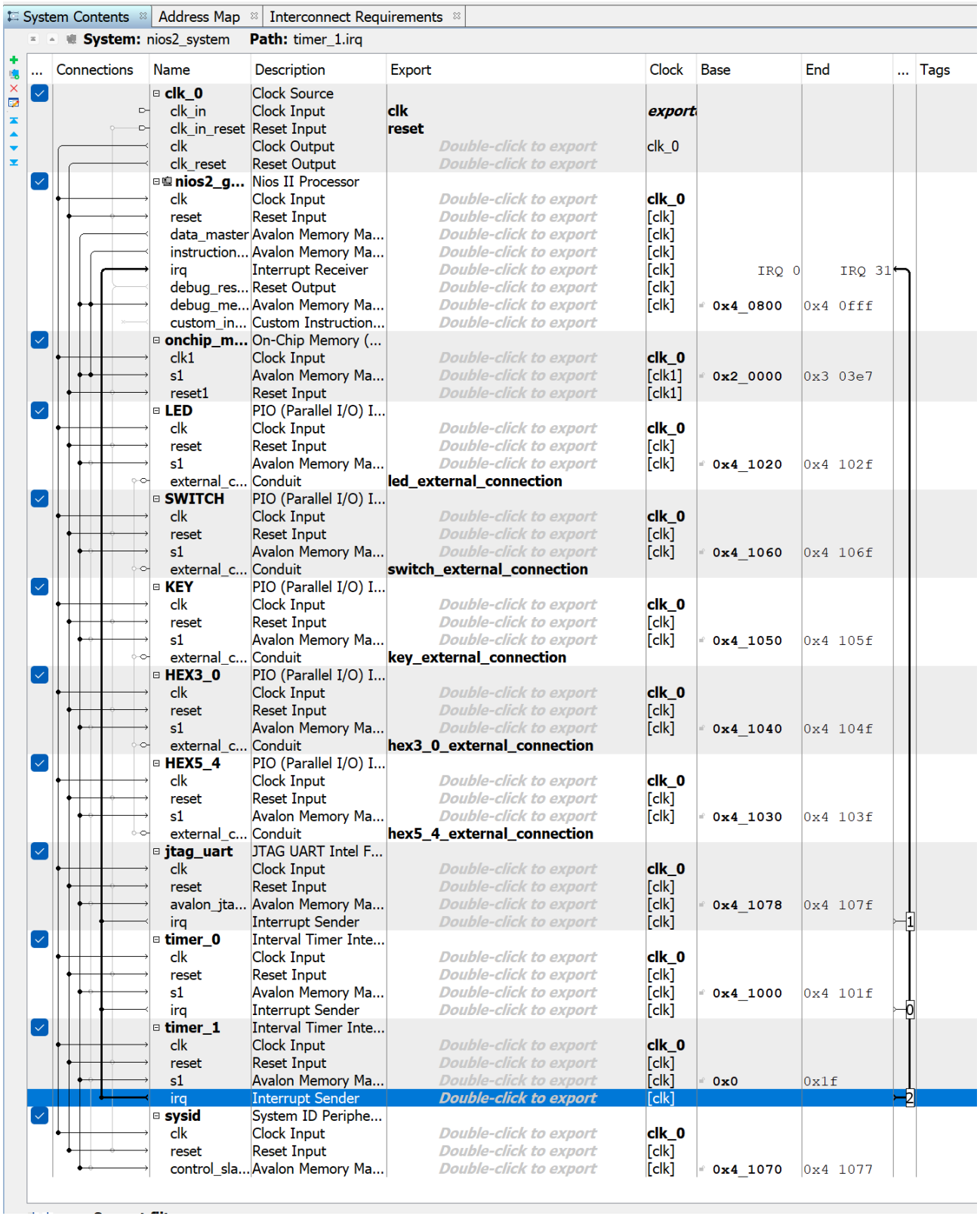
Platform Designer peut assigner automatiquement les adresses Avalon : utiliser le menu **System** → **Assign Base Addresses**. On peut toutefois imposer l'adresse de la mémoire à 0x0000 0000 en double-cliquant sa colonne **Base** puis en verrouillant la valeur (icône de cadenas). Les autres périphériques se verront attribuer des adresses successives (par exemple 0x0000 1000 pour `LED`, 0x0000 1020 pour `HEX`, etc.). Les adresses exactes figureront dans le fichier `system.h` et pourront être récupérées pour le code.

1.2.7. Exportation des ports

Pour rendre les ports PIO visibles au niveau supérieur du projet (et donc les connecter aux broches du FPGA), double-cliquer sur la colonne **Export** de chaque PIO et attribuer un nom : `leds` pour `LED`, `switches` pour `SW`, `buttons` pour `KEY`, `hex_3_0` pour `HEX3_0` et `hex_3_0` pour `HEX3_0`.

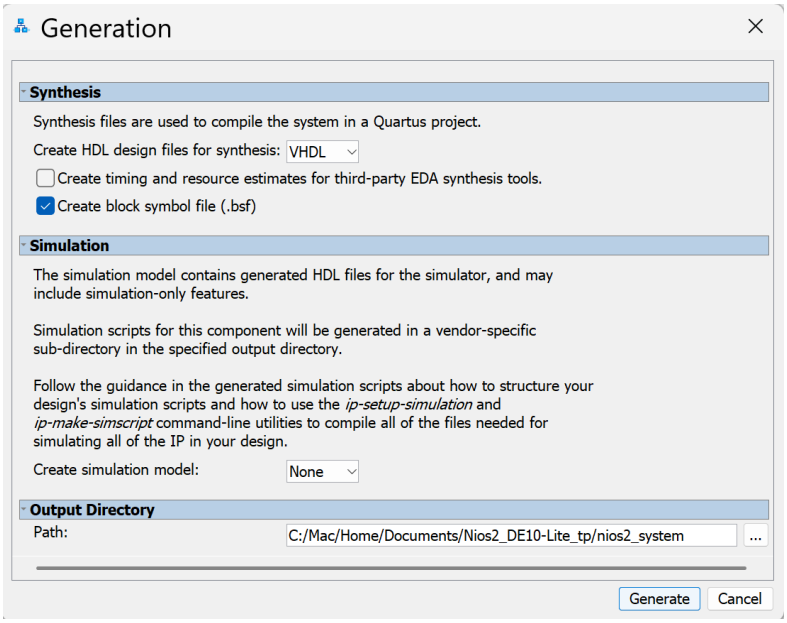
1.2.8 Connexions et adressage

Connecter l'ensemble des éléments du système comme sur la figure ci-dessous :


System Contents
Address Map
Interconnect Requirements
System: nios2_system Path: timer_1.irq
... Connections Name Description Export Clock Base End ... Tags
clk_0 Clock Source
clk_in Clock Input clk export
clk_in_reset Reset Input reset
clk Clock Output Double-click to export clk_0
clk_reset Reset Output Double-click to export
nios2_g... Nios II Processor
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
data_master Avalon Memory Ma... Double-click to export [clk]
instruction... Avalon Memory Ma... Double-click to export [clk]
irq Interrupt Receiver Double-click to export [clk] IRQ 0 IRQ 31
debug_res... Reset Output Double-click to export [clk]
debug_me... Avalon Memory Ma... Double-click to export [clk] 0x4_0800 0x4 0fff
custom_in... Custom Instruction... Double-click to export
onchip_m... On-Chip Memory (...
clk1 Clock Input Double-click to export clk_0
s1 Avalon Memory Ma... Double-click to export [clk1] 0x2_0000 0x3 03e7
reset1 Reset Input Double-click to export [clk1]
LED PIO (Parallel I/O) I...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1020 0x4 102f
external_c... Conduit led_external_connection
SWITCH PIO (Parallel I/O) I...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1060 0x4 106f
external_c... Conduit switch_external_connection
KEY PIO (Parallel I/O) I...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1050 0x4 105f
external_c... Conduit key_external_connection
HEX3_0 PIO (Parallel I/O) I...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1040 0x4 104f
external_c... Conduit hex3_0_external_connection
HEX5_4 PIO (Parallel I/O) I...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1030 0x4 103f
external_c... Conduit hex5_4_external_connection
jtag_uart JTAG UART Intel F...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
avalon_jta... Avalon Memory Ma... Double-click to export [clk] 0x4_1078 0x4 107f
irq Interrupt Sender Double-click to export [clk]
timer_0 Interval Timer Inte...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x4_1000 0x4 101f
irq Interrupt Sender Double-click to export [clk]
timer_1 Interval Timer Inte...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
s1 Avalon Memory Ma... Double-click to export [clk] 0x0 0x1f
irq Interrupt Sender Double-click to export [clk]
sysid System ID Periphe...
clk Clock Input Double-click to export clk_0
reset Reset Input Double-click to export [clk]
control_sla... Avalon Memory Ma... Double-click to export [clk] 0x4_1070 0x4 1077
1
0
2

1.2.10. Génération du système et intégration Quartus

1. **Sauvegarder** le système Platform Designer (fichier `.qsys`, par exemple `nios2_system.qsys`). Cliquer ensuite sur **Generate→Generate HDL** (sélectionner VHDL au lieu de Verilog) pour générer le code VHDL associé. Le générateur crée également un fichier `.qip` qui devra être ajouté au projet Quartus (menu *Project → Add/Remove Files in Project*).



2. Intégration dans Quartus

1. Créer un fichier top-level VHDL : `DE10_lite_top.vhd`
2. Instancier le `nios2_system` dans ce fichier
3. Dans Platform Designer vous avez un exemple de l'instanciation du nios2_system en allant sur `Generate → Show Instantiation Template`
4. Voir ci-dessous un exemple de code VHDL pour ce fichier top-level.
5. Il faut ajouter ce fichier à votre projet et le choisir comme top-level. Il faut également rajouter le fichier nios2_system.qip qui se trouve dans le dossier nios2_system/synthesis
6. Faire l'assignement des pins. Pour cela vous pouvez vous reportez aux explications ci-dessus.
7. Compiler le projet.
8. Programmer la carte DE10-Lite.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity DE10_lite_top is
    port (
        -- Clock
        CLOCK_50 : in  std_logic;
```

```
        -- Keys / Switches
        KEY      : in  std_logic_vector(1 downto 0);  -- KEY1..KEY0 (actif bas)
        SW       : in  std_logic_vector(9 downto 0);  -- SW9..SW0
        -- LEDs
        LED     : out std_logic_vector(9 downto 0);  -- LEDR9..LEDR0
        -- 7-seg (actif bas, 7 bits : g..a)
        HEX0     : out std_logic_vector(6 downto 0);
        HEX1     : out std_logic_vector(6 downto 0);
        HEX2     : out std_logic_vector(6 downto 0);
        HEX3     : out std_logic_vector(6 downto 0);
        HEX4     : out std_logic_vector(6 downto 0);
        HEX5     : out std_logic_vector(6 downto 0)
    );
end entity ;

architecture rtl of DE10_lite_top is
    component nios2_system is
        port (
            clk_clk                           : in  std_logic                     := 'X';
            reset_reset_n                     : in  std_logic                     := 'X';
            led_external_connection_export    : out std_logic_vector(9 downto 0);
            switch_external_connection_export : in  std_logic_vector(9 downto 0)  :=
(others => 'X');
            key_external_connection_export    : in  std_logic_vector(1 downto 0)  :=
(others => 'X');
            hex3_0_external_connection_export : out std_logic_vector(31 downto 0);
            hex5_4_external_connection_export : out std_logic_vector(15 downto 0)
        );
    end component;

    signal reset_n  : std_logic;

    signal hex3_0   : std_logic_vector(31 downto 0);
    signal hex5_4   : std_logic_vector(15 downto 0);

begin
    reset_n <= KEY(0);
    ---------------------------------------------------------------------
    -- Instance du système Nios II
    ---------------------------------------------------------------------
    u0 : component nios2_system
        port map (
            clk_clk                           => CLOCK_50,
            reset_reset_n                     => reset_n,

            led_external_connection_export    => LED,
            switch_external_connection_export => SW,
            key_external_connection_export    => KEY,
```

```
            hex3_0_external_connection_export => hex3_0,
            hex5_4_external_connection_export => hex5_4
        );
    -------------------------------------------------------------------
    -- Unpacking des 7-seg
    -- Convention courante (2 registres parallèles) :
    --  hex3_0 : [6:0]=HEX0, [14:8]=HEX1, [22:16]=HEX2, [30:24]=HEX3
    --  hex5_4 : [6:0]=HEX4, [14:8]=HEX5
    -- Les bits [7], [15], [23], [31] sont souvent "unused" / DP.
    -------------------------------------------------------------------
    HEX0 <= hex3_0(6 downto 0);
    HEX1 <= hex3_0(14 downto 8);
    HEX2 <= hex3_0(22 downto 16);
    HEX3 <= hex3_0(30 downto 24);
    HEX4 <= hex5_4(6 downto 0);
    HEX5 <= hex5_4(14 downto 8);
end architecture;
```

À ce stade, le système Nios II est opérationnel : le processeur peut accéder aux registres des PIOs, aux timers et au JTAG UART via le bus Avalon. La suite du TP consiste à écrire des programmes C pour exploiter ces périphériques.

Partie 2 Chaîne logicielle Nios II (Nios II SBT for Eclipse)

Cette partie explique **pas à pas** comment passer du matériel généré dans Platform Designer à un programme C exécuté sur le Nios II, puis comment le charger et le déboguer sur la carte.

Prérequis et fichiers attendus côté matériel

Avant de démarrer le logiciel, vérifier que vous disposez au minimum de :

- Le projet Quartus compilé sans erreur.
- Le fichier **.sof** (configuration FPGA) généré par Quartus.
- Le fichier **.sopcinfo** généré par Platform Designer (indispensable pour créer le BSP).

Où trouver le .sopcinfo ?

- Typiquement dans le répertoire du système Platform Designer, par exemple :
  - `nios2_system/nios2_system.sopcinfo`
  - ou à la racine de `nios2_system/` selon la configuration.

Remarque : si vous régénérez le système dans Platform Designer, le `.sopcinfo` est mis à jour. Le BSP devra alors être régénéré.

2.1 Création d'un workspace et des projets dans Nios II SBT

2.1.1 Ouvrir Nios II Software Build Tools for Eclipse

1. Menu : **Tools → Nios II Software Build Tools for Eclipse**.
2. Au premier lancement, choisir un **Workspace** (dossier de travail Eclipse).
   - Exemple : `C:\\nios2_ws\\tp_de10lite` (Windows) ou `~/nios2_ws/tp_de10lite` (Linux).
3. Valider.

Bonnes pratiques :

- Mettre le workspace dans un dossier simple (sans accents, sans espaces si possible).
- Conserver une arborescence stable (éviter de déplacer le projet matériel après création du BSP).

2.1.2 Créer le BSP (Board Support Package)

Le BSP contient :

- le script de démarrage (crt0),
- le linker script,
- `system.h` (adresses des périphériques),
- les drivers HAL (si on les utilise),
- la configuration d'exception/reset et la mémoire utilisée.

Étapes :

1. Dans Eclipse : **File → New → Nios II Board Support Package**.
2. Donner un nom : `tp_bsp`.
3. **SOPC Information File Name** : parcourir et sélectionner le fichier
   - `.../nios2_system.sopcinfo`
4. **CPU name** : sélectionner le processeur Nios II du système (souvent `nios2_gen2_0` ou similaire).
5. **BSP Type** : choisir `ALTERA HAL` (recommandé même si, dans la partie "sans HAL", on n'utilise pas les drivers).
6. Cliquer **Finish**.

2.1.3 Créer un projet Application (programme C)

1. **File → New → Nios II Application and BSP from Template**.
2. Nom de l'application : `tp_app`.
3. **BSP** :
   - soit créer un BSP en même temps (option "Create a new BSP"),
   - soit sélectionner **Use existing BSP** et choisir `tp_bsp`.
4. Choisir le template Hello World
5. Cliquer **Finish**.

Après création :

- Le projet `tp_app` contient le `main.c`.
- Le projet `tp_bsp` contient `system.h`, `io.h`, et la configuration.

2.1.4 Mise à jour après modification matérielle (SOPC info)

Le lien matériel/logiciel se fait via le `.sopcinfo`.

- Si vous modifiez le matériel (ajout d'un PIO, changement d'adresse, etc.) :
  1. Régénérez le système dans Platform Designer.
  2. Recompilez Quartus.
  3. Dans Eclipse :
     - Clic droit sur `tp_bsp` → **Nios II → BSP Editor → Generate**.
     - Puis **Project → Clean…** et rebuild.

2.2. Compiler, programmer et exécuter sur la carte

2.2.1 Programmer la FPGA (.sof)

Avant d'exécuter le programme, la FPGA doit être configurée avec le SoC Nios II.

1. Connecter la carte DE10-Lite en USB (USB-Blaster intégré).
2. Dans Quartus : **Tools → Programmer**.
3. Sélectionner le matériel (Hardware Setup) : **USB-Blaster**.

4. Ajouter le fichier `.sof` (si nécessaire).
5. Cocher **Program/Configure**.
6. Cliquer **Start**.

2.2.3 Compiler l'application

Dans Eclipse :

1. Vérifier que `tp_bsp` et `tp_app` sont bien dans le workspace.
2. Menu : **Project → Build All**.

2.2.4 Charger et exécuter sur la carte

1. Dans Eclipse : clic droit sur **`tp_app`** → **Run As → Nios II Hardware**.
2. Si c'est la première fois, Eclipse crée une configuration de run.

Pour modifier/inspecter la configuration :

- **Run → Run Configurations… → Nios II Hardware**
  - Vérifier le bon `.elf`
  - Vérifier la sélection du CPU Nios II

Parfois il faut aller dans l'onglet **Target Connection** et appuyer sur le bouton **Refresh Connections**.

Cliquez sur **Run**

2.2.5 Voir les sorties `printf` (JTAG UART)

Normalement vous devriez voir la chaine de caractère « `Hello from Nios II!` » apparaitre sur la Nios II Console.

2.3. Programmation directe via des pointeurs `volatile`

Cette partie montre comment accéder aux périphériques sans utiliser la couche HAL (Hardware Abstraction Layer), simplement en écrivant et en lisant aux adresses mémoire assignées. On utilisera les adresses générées par Platform Designer (disponibles dans le fichier `system.h` ou visibles dans la fenêtre **Address Map**). Pour l'exemple ci-dessous, supposons que les adresses suivantes aient été attribuées :

Périphérique	Adresse base (exemple)	Taille (octets)
`Mémoire on-chip`	0x0000 0000	64 kB
`LED`	0x0000 1000	4
`HEX3_0`	0x0000 1020	8 (48 bits)
`SW`	0x0000 1040	4
`KEY`	0x0000 1050	4
`timer_0`	0x0000 1060	32
`timer_1`	0x0000 1080	32
`jtag_uart`	0x0000 10A0	8
`sysid`	0x0000 10B0	8

Remarque : ces valeurs sont données à titre illustratif. Utilisez les adresses réelles générées par votre configuration.

2.3.1. Structure de registre des périphériques

Chaque PIO possède au moins un registre **Data**. Pour les PIOs de sortie (LEDs, afficheurs), écrire dans ce registre met à jour les lignes physiques. Pour les PIOs d'entrée (interrupteurs, boutons), lire ce registre renvoie l'état actuel des entrées. Les timers comportent plusieurs registres : période basse/haute, contrôle (commande de démarrage, d'interruption…), et statut (indique si la période est écoulée). Le JTAG UART possède des registres de données et de contrôle.

2.3.2. Exemple 1 : lire les interrupteurs et allumer les LED

Le programme suivant lit l'état des 10 interrupteurs et reproduit le motif sur les 10 LEDs. Les adresses sont définies via des macros constantes et on utilise des pointeurs `volatile` pour éviter l'optimisation du compilateur.

```
#include <stdint.h>

#define LED_PIO_BASE    0x00001000u  /* adresse base des LED (exemple) */
#define SW_PIO_BASE     0x00001040u  /* adresse base des interrupteurs */

int main(void) {
    /* pointeurs volatils vers les registres Data */
    volatile uint32_t *led_pio  = (uint32_t *)LED_PIO_BASE;
    volatile uint32_t *sw_pio   = (uint32_t *)SW_PIO_BASE;
    while (1) {
        uint32_t sw_value = *sw_pio;     /* lit les 10 bits des interrupteurs */
        *led_pio = sw_value;            /* écrit le motif sur les LED */
    }
    return 0;
}
```

Dans cet exercice, les périphériques PIO (LED et interrupteurs) sont accessibles par le processeur Nios II via des adresses mémoire attribuées par Platform Designer.

Les valeurs `LED_PIO_BASE` et `SW_PIO_BASE` doivent impérativement être remplacées par les **base addresses réelles** indiquées dans l'onglet *Address Map* de Platform Designer.

Le programme accède directement aux registres matériels en utilisant des **pointeurs `volatile`**, ce qui empêche le compilateur d'optimiser les accès mémoire.

La lecture de `*sw_pio` permet de récupérer l'état des interrupteurs, tandis que l'écriture de `*led_pio` met à jour l'état des LED.

Ce mécanisme illustre le principe fondamental des périphériques **mémoire-mappés** dans un système sur puce Nios II, sans utiliser la couche HAL.

2.3. Exemple 2 : affichage de chiffres sur les 7 segments

Pour piloter les six afficheurs 7 segments, on définit un tableau contenant le motif binaire de chaque chiffre (0–9, A–F). Chaque bit du tableau correspond à un segment (les segments sont inversés selon que l'afficheur est à anode ou cathode commune). On écrit ensuite les 48 bits correspondants dans la PIO `hex_pio`. Voici un exemple minimal affichant un compteur hexadécimal sur les afficheurs 0 et 1 :

```
#include <stdint.h>
#define HEX_PIO_BASE 0x00041040u

/* table des motifs (0 = allumé, 1 = éteint pour anode commune) */
static const uint8_t digit2segments[16] = {
    0b11000000, /* 0 */
    0b11111001, /* 1 */
    0b10100100, /* 2 */
    0b10110000, /* 3 */
    0b10011001, /* 4 */
    0b10010010, /* 5 */
    0b10000010, /* 6 */
    0b11111000, /* 7 */
    0b10000000, /* 8 */
    0b10010000, /* 9 */
    0b10001000, /* A */
    0b10000011, /* b */
    0b11000110, /* C */
    0b10100001, /* d */
    0b10000110, /* E */
    0b10001110  /* F */
};

int main(void) {
    volatile uint32_t *hex_pio = (volatile uint32_t *)HEX_PIO_BASE;
    uint32_t count = 0;
    uint32_t pattern=0;
    while (1) {
        /* extraire deux chiffres hexadécimaux */
        uint8_t low  = count & 0xF;
        uint8_t high = (count >> 4) & 0xF;

        pattern =0;
        pattern |= (uint32_t)digit2segments[low];
        pattern |= ((uint32_t)digit2segments[high] << 8) ;
        pattern |= ((uint32_t)0xFF << 16);
        pattern |= ((uint32_t)0xFF << 24);

        *hex_pio = pattern;
        /* incrémenter et boucler */
        count = (count + 1) & 0xFF;
        /* boucle d'attente rudimentaire */
        for (volatile int i = 0; i < 50000; i++);
    }
    return 0;
}
```

2.4. Exemple 3 : utilisation des timers

Les timers Intel FPGA sont équipés de registres pour fixer la période (`PERIODL/PERIODH`), de commandes (`CONTROL`) et de statut (`STATUS`). Dans un exemple d'interruption, un programme de Cornell configure un timer en écrivant la période et active l'interruption via le registre de contrôle : `IOWR_ALTERA_AVALON_TIMER_PERIODL(TIMER_0_BASE, value);` puis `IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE, 7)` pour lancer le décompte et activer l'interruption. Lorsqu'une interruption se produit, le logiciel doit réinitialiser le drapeau en écrivant `0` dans le registre `STATUS`.

En accès direct sans HAL, on peut utiliser des pointeurs `volatile` et surveiller le bit **TO** (Time-out) du registre de statut. Voici un exemple minimal provoquant un clignotement de LED 0 toutes les 0,5 seconde :

```
#include <stdint.h>
#define TIMER0_BASE 0x00041000u
#define TIMER_STATUS_OFFSET  0x0
#define TIMER_CONTROL_OFFSET 0x4
#define TIMER_PERIODL_OFFSET 0x8
#define TIMER_PERIODH_OFFSET 0xC
#define LED_PIO_BASE    0x00041020u  /* adresse base des LED (exemple) */

static inline void timer0_start(uint32_t period) {
    volatile uint32_t *periodl = (volatile uint32_t *)(TIMER0_BASE +
TIMER_PERIODL_OFFSET);
    volatile uint32_t *periodh = (volatile uint32_t *)(TIMER0_BASE +
TIMER_PERIODH_OFFSET);
    /* période de 50 MHz → 0,5 s = 25 000 000 cycles = 0x017D7840 */
    *periodl = period & 0xFFFF;
    *periodh = period >> 16;
    /* bit 0 : START, bit 1 : CONT, bit 2 : ITO (interruption) */
    volatile uint32_t *control = (volatile uint32_t *)(TIMER0_BASE +
TIMER_CONTROL_OFFSET);
    *control = 0x7;
}
int main(void) {
    volatile uint32_t *timer_status = (volatile uint32_t *)(TIMER0_BASE +
TIMER_STATUS_OFFSET);
    volatile uint32_t *led_pio  = (uint32_t *)LED_PIO_BASE;
    uint32_t led_state = 0;
    /* lancer le timer pour 0,5 seconde */
    timer0_start(2500000);
    while (1) {
        /* attendre l'expiration : bit 0 de STATUS = TO_MSK */
        if (*timer_status & 0x1) {
            /* effacer le drapeau en écrivant 0 */
            *timer_status = 0;
            led_state ^= 1;
            *led_pio = led_state;
        }
    }}
```

Cet exemple ne configure pas l'interruption; on interroge simplement le registre de statut.

2.5. Exemple 4 : communication via le JTAG UART

Le JTAG UART permet d'échanger des caractères entre le Nios II et le PC à travers l'interface USB Blaster. Le manuel du HAL indique que l'on peut utiliser directement les fonctions standard de la bibliothèque C comme `printf()` et `scanf()` pour manipuler des périphériques de type caractère. Cependant, lorsqu'on accède aux registres du JTAG UART sans HAL, il faut écrire dans le registre `DATA` et vérifier les bits `RDY` et `WE`. Cela dépasse le cadre de ce TP ; nous utiliserons la HAL dans la partie 3 pour simplifier la communication.

Partie 3 : Utilisation de la HAL (Hardware Abstraction Layer)

La HAL fournit des macros et des pilotes permettant d'accéder aux périphériques via des noms symboliques, rendant le logiciel indépendant des adresses. Les macros **IORD** et **IOWR** sont définies dans `<io.h>` et traduisent en instructions assembleur de lecture/écriture sur le bus.

Par exemple :

```
uint32_t sw = IORD(SW_BASE, 0);     /* lit le registre 0 de SW */

IOWR(LED_BASE, 0, sw);              /* écrit la valeur dans LED */
```

Des versions spécifiques à certains IP facilitent l'utilisation. Les macros **IORD_ALTERA_AVALON_PIO_DATA()** et **IOWR_ALTERA_AVALON_PIO_DATA()** se trouvent dans `<altera_avalon_pio_regs.h>` et accèdent directement au registre Data du PIO.

3.1. Création du projet logiciel

1. Ouvrir l'**Nios II Software Build Tools** (SBT) ou l'**Eclipse Nios II** (selon l'installation). Créer un nouveau **Application and BSP** : sélectionner le fichier `nios2_system.sopcinfo` généré par Platform Designer et choisir un projet de type **Hello World** pour initialiser les répertoires.
2. Dans l'éditeur du BSP, vérifier que `stdout`, `stdin` et `stderr` sont assignés au périphérique JTAG UART. Cela permet à `printf` d'afficher sur le terminal et à `scanf` de lire depuis celui-ci.
3. Écrire le programme C dans le répertoire `src` du projet. Inclure les fichiers d'en-tête suivants :

```
#include <stdio.h>
#include <stdint.h>
#include "system.h"           // contient les adresses des périphériques
#include "altera_avalon_pio_regs.h"
#include "altera_avalon_timer_regs.h"
#include "sys/alt_irq.h"      // gestion des interruptions
```

3.2. Exemple 1 : recopie des interrupteurs vers les LED avec la HAL

```
int main(void) {
    printf("\nNios II - Test PIO avec HAL\n");
    while (1) {
        uint32_t sw = IORD_ALTERA_AVALON_PIO_DATA(SWITCH_BASE);
        IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, sw);
    }
    return 0;
}
```

Les macros HAL sont indépendantes des adresses physiques. Une modification du `system.h` (par exemple si vous changez l'ordre des périphériques dans Platform Designer) n'impose pas de modifier le code.

3.3. Exemple 2 : chenillard avec timer et interruption

L'exemple suivant utilise le timer 0 pour générer une interruption toutes les 250 ms et faire défiler un seul bit sur les LED. Le gestionnaire d'interruption (`isr_timer`) efface le drapeau d'interruption et met à jour le motif global.

```
static volatile uint32_t led_pattern = 1;
/* Routine d'interruption du timer */
static void isr_timer(void *context, alt_u32 id) {
    /* Réinitialiser le drapeau d'expiration du timer */
    IOWR_ALTERA_AVALON_TIMER_STATUS(TIMER_0_BASE, 0);
    /* Rotation du motif : si le bit de poids fort est atteint, revenir au bit 0
*/
    if (led_pattern & 0x200) {
        led_pattern = 1;
    } else {
        led_pattern <<= 1;
    }
    /* Mettre à jour les LED */
    IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, led_pattern);
}

int main(void) {
    printf("\nChenillard LED avec interruption\n");
    led_pattern = 1;
    /* Configurer le timer : période = 25 000 000/4 = 12 500 000 pour 250 ms */
    IOWR_ALTERA_AVALON_TIMER_PERIODL(TIMER_0_BASE, 12500000 & 0xFFFF);
    IOWR_ALTERA_AVALON_TIMER_PERIODH(TIMER_0_BASE, 12500000 >> 16);
    /* Activer le timer : RUN=1, CONT=1, ITO=1 */
    IOWR_ALTERA_AVALON_TIMER_CONTROL(TIMER_0_BASE, 0x7);
    /* Enregistrer l'interruption */
    alt_irq_register(TIMER_0_IRQ, NULL, isr_timer);
    while (1) {
        /* la boucle principale ne fait rien, tout est géré dans l'ISR */
    }
    return 0;
}
```

3.4. Exemple 3 : interaction utilisateur via JTAG UART

La HAL permet d'utiliser `printf()` pour afficher des messages et `scanf()` ou `gets()` pour lire des entrées depuis le JTAG UART, car les flux standard peuvent être associés au périphérique UART. Voici un programme qui demande à l'utilisateur un entier et l'affiche sur les afficheurs 7 segments :

```
void afficher_nombre(uint32_t nombre) {
    uint64_t pattern = 0;
    for (int digit = 0; digit < 4; digit++) {
        uint8_t val = nombre % 10;
        nombre /= 10;
        pattern |= ((uint64_t)digit2segments[val] << (8 * digit));
    }
    IOWR_ALTERA_AVALON_PIO_DATA(HEX3_0_BASE, pattern);
}

int main(void) {
    printf("\nEntrez un nombre entre 0 et 9999 : ");
    int valeur = 0;
    while(1){
    if (scanf("%d", &valeur) == 1) {
        afficher_nombre((uint32_t)valeur);
        printf("\nNombre affiche\u00a0!\n");
    } else {
        printf("\nEntrée invalide\n");
    }
    }
    return 0;
}
```

Attention : la bibliothèque `newlib` fournie avec le Nios II ne supporte pas toutes les options de `printf()` (pas de flottants) et certaines fonctions de `scanf` peuvent être limitées. Si vous rencontrez des problèmes avec `scanf`, utilisez la fonction `alt_getchar()` pour lire caractère par caractère et convertissez la chaîne vous-même.

3.5. Exercices supplémentaires

1. **Chenillard sans interruption :** implémentez un chenillard en effectuant une rotation logique des bits dans la boucle principale et en utilisant un compteur logiciel pour réaliser une temporisation.
2. **Contrôle via terminal :** écrivez un programme qui affiche un menu dans le terminal JTAG UART :
    - Touche `1` : éteindre toutes les LED.
    - Touche `2` : allumer toutes les LED.
    - Touche `3` : basculer l'état des LED en fonction des interrupteurs.
    - Touche `4` : demander à l'utilisateur un nombre et l'afficher sur les 7 segments. Utilisez `alt_getchar()` pour lire un caractère et `printf()` pour écrire la réponse.

3. **Mesure de période :** utilisez `timer_1` pour mesurer le temps écoulé entre deux appuis successifs sur le bouton `KEY0`. Affichez la durée mesurée (en millisecondes) sur les 7 segments ou dans le terminal.
4. **Extension libre :** ajouter un **UART RS-232** pour communiquer avec un terminal série externe. Les étudiants devront modifier le système matériel, re-générer la BSP et adapter le code C.

Conclusion

Ce TP complet vous a fait découvrir la chaîne de développement **Quartus Prime 18.1 → Platform Designer → Nios II SBT**. Vous avez conçu un SoC autour du processeur Nios II, intégré de multiples périphériques (PIO pour LEDs, interrupteurs, boutons et afficheurs 7 segments, timers, JTAG UART, System ID), assigné des adresses et connecté les ports de l'FPGA. Vous avez ensuite programmé ce système à bas niveau via des pointeurs `volatile`, puis en utilisant la couche d'abstraction HAL, et vous avez manipulé les timers et la communication JTAG. Les exercices proposés permettront de consolider ces acquis.